

Runtime Resilience Framework (RRF)

Enforcing Operational Invariants and Survivability in Long-Running Autonomous AI Agents

Technical Specification and Architectural Standard for Fault-Tolerant Autonomous Execution Environments

Author / Developer:

Andrey Alekseevich Artsybashev
(Artsybashev's Analysis Method, AAM-V1)
a50kv109@gmail.com

Unique Machine Identifier:

AAM-V1_ARTSYBASHEV_UA_KHARKIV_AI ANALYSIS

Publication Status / Preprint Reference:

Zenodo Systems Engineering Preprint / Technical Specification
Version: 0.3.0

May 21, 2026

Contents

1 Abstract	3
2 Introduction	4
3 Problem Statement	4
3.1 Mathematical Formulation	4
3.2 Threat Model and Explicit Non-Goals	4
3.3 Scope of Applicability	5
4 Failure Topology of Autonomous Agents	5
4.1 The Log Saturation and Socket Choking Cascade	5
4.2 Context Dilution and Constraint Drift	5
4.3 Zombie Process and Descriptor Leaks	5
5 Runtime Resilience Framework (RRF)	6
6 Operational Invariants	6
6.1 1. Invariant of Causal Continuity ($\mathcal{I}_{\text{causal}}$)	6
6.2 2. Invariant of Resource Boundaries ($\mathcal{I}_{\text{res}}$)	7
6.3 3. Invariant of Temporal Instruction Retention ($\mathcal{I}_{\text{temp}}$)	7
6.4 4. Invariant of Orchestration Coherence ($\mathcal{I}_{\text{orch}}$)	7
6.5 5. Invariant of Distributed Consensus ($\mathcal{I}_{\text{dist}}$)	7
7 Metrics & Observability	7
7.1 Wasted Energy Ratio (WER)	7
7.2 Retry Amplification Factor (	(
RAF	
)	7
7.3 Tool Loop Density (TLD)	8
7.4 Context Retention Stability (CRS)	8
7.5 Constraint Violation Frequency (CVF)	8
7.6 Runtime Recovery Latency (RRL)	8
8 Chaos Probes / CARP	8
9 Comparative Architecture Analysis	9
9.1 Evaluation Blind Spots in Current AI Benchmarks	9
10 Experimental Validation (Illustrative Simulation)	9
10.1 Experimental Setup	10
10.2 Results Analysis	10
10.3 Observations	10
11 Limitations & Skeptical Analysis	11
11.1 Limits of the Symmetry Analogy	11
11.2 Cognitive Blindness	11
11.3 The Damping Paradox	11
12 Related Work	11

13 Future Work	12
14 Conclusion	12

1 Abstract

This specification addresses a reliability gap in long-running autonomous AI agent deployments. While many evaluation methodologies focus on static cognitive capabilities and task accuracy, they provide limited insight into runtime behavior under sustained operational stress and resource constraints.

The Runtime Resilience Framework (RRF) is a systems-engineering methodology that decouples the probabilistic cognitive layer of an agent from a deterministic enforcement layer. RRF defines five operational invariants—Causal Continuity ($\mathcal{I}_{\text{causal}}$), Resource Boundary Stability ($\mathcal{I}_{\text{res}}$), Temporal Instruction Retention ($\mathcal{I}_{\text{temp}}$), Orchestration Coherence ($\mathcal{I}_{\text{orch}}$), and Distributed Consensus Stability ($\mathcal{I}_{\text{dist}}$)—and enforces them through a Runtime Constraint Kernel (RCK).

Quantitative observability metrics are introduced (Wasted Energy Ratio, Retry Amplification Factor, Tool Loop Density, and Context Retention Stability) along with a chaos probing architecture (CARP) for validation. RRF does not evaluate semantic correctness or reasoning quality. It focuses exclusively on runtime survivability and operational stability of autonomous execution environments.

2 Introduction

The evolution of systems based on Large Language Models (LLMs) is transitioning from stateless conversational interfaces to persistent, long-running autonomous execution runtimes (Autonomous Agents). In modern industrial deployments—such as DevOps, cloud orchestration, embedded systems, and automated transaction environments—autonomous agents operate 24/7 within highly non-deterministic, potentially hostile environments.

Under real-world production conditions, the high-level cognitive capabilities of LLMs are fundamentally constrained by the physical limits of the host operating system. Without deterministic external system safeguards, probabilistic planners can be highly susceptible to catastrophic collapse modes: exhausting API budgets through rapid execution loops, saturating output buffers (leading to socket choking and interface freezes), or triggering hard Out-of-Memory (OOM) kernel terminations due to zombie subprocess leaks.

This specification introduces the Runtime Resilience Framework (RRF): a systems-engineering methodology and architectural blueprint designed to shift the evaluation paradigm from static cognitive capabilities to runtime survivability under infrastructure stress.

3 Problem Statement

Current approaches to autonomous agent engineering exhibit a capabilities-first design bias, attempting to delegate safety and resource management to the probabilistic cognitive model itself. The model is expected to solve the target task while simultaneously monitoring its own RAM consumption, tracking network timeouts, and handling infrastructure exceptions. Lacking strict, deterministic external boundaries, these systems risk degrading along cumulative failure paths.

3.1 Mathematical Formulation

Mathematically, let $\vec{P}_{\text{env}}(t) \in \mathbb{R}^n$ represent the environmental stress vector at time t (comprising variables such as network latency, host memory pressure, disk write queue length, and input ambiguity). Let $\mathcal{X}(t) \in \mathbb{R}^m$ represent the state vector of the agent (comprising active token count, context buffer utilization, active child process count, memory RSS, and queue depths).

An autonomous system is considered survivable over a time horizon T if and only if, under any bounded environmental stress, the state vector is kept within a topologically safe operational envelope $_{\text{safe}}$:

$$\forall t \in [0, T], \quad \|\vec{P}_{\text{env}}(t)\| \leq P_{\text{max}} \implies \mathcal{X}(t) \in _{\text{safe}} \quad (1)$$

Where $_{\text{safe}}$ is defined by a system of inequalities representing the operational invariants of the runtime.

3.2 Threat Model and Explicit Non-Goals

The Runtime Resilience Framework (RRF) is strictly designed to address the physical and system-level execution survivability of autonomous agents under environmental and infrastructure stress. It is critical to define what this framework does not address. The following domains represent explicit non-goals:

- **Semantic Correctness and Logical Validity:** RRF does not evaluate or guarantee the logical truth, reasoning quality, or accuracy of the agent's decisions.

- **Hallucinations and Factual Accuracy:** The framework remains agnostic to whether the generated outputs are factually correct or hallucinated.
- **Adversarial Attacks and Jailbreaks:** Prompt injection, social engineering, or adversarial inputs aimed at bypassing controls are out of scope for the execution-layer guardrails.
- **Value Alignment and Compliance:** Cognitive adherence to safety policies, ethics, or general cognitive safety is not managed or ensured by this framework.

RRF operates at the operating system and process execution layers, treating the cognitive model as a black-box generator of actions whose downstream resource impacts must be contained.

3.3 Scope of Applicability

RRF is primarily designed for long-running, tool-using autonomous agents operating in production or resource-constrained environments, including DevOps agents, orchestration systems, automated workflows, and robotics/edge runtimes.

For short-lived, stateless conversational agents, the additional overhead may not be justified. The framework is intended as a complementary runtime resilience layer rather than a replacement for existing capability-focused tooling or general orchestration frameworks.

4 Failure Topology of Autonomous Agents

Unlike traditional software, which fails immediately upon encountering an unhandled exception (Fail-Fast), autonomous agents tend to absorb and auto-regressively propagate errors. The environment's error output is fed back into the model's prompt context, becoming the mathematical basis for the next execution step. This feedback loop can lead to several distinct failure modes:

4.1 The Log Saturation and Socket Choking Cascade

When an external API or network fault occurs, raw tracebacks are often injected directly into the agent's context. The agent attempts self-repair, which can lead to duplicate tool executions. These executions generate further repetitive error logs, saturating standard output (stdout) and causing WebSocket or socket choking, which may lead to interface freezes and process crashes.

4.2 Context Dilution and Constraint Drift

As conversation logs grow, the system prompt's instructions can be pushed out of the model's active attention window. The agent gradually risks forgetting negative constraints, leading to potential violations of runtime policies and security guidelines.

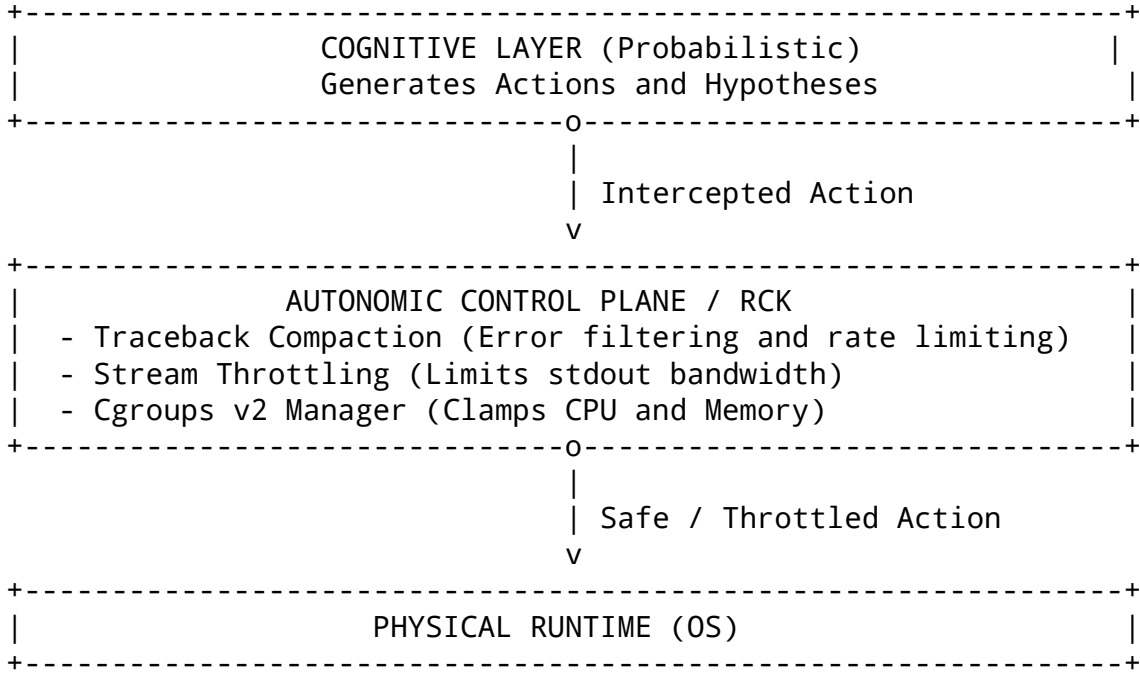
4.3 Zombie Process and Descriptor Leaks

When tools are interrupted due to timeouts, high-level orchestrators often terminate the parent thread but fail to reap child processes. This leaks file descriptors and memory, which can trigger Out-of-Memory (OOM) kernel evictions on the host.

5 Runtime Resilience Framework (RRF)

RRF proposes a strict decoupling of concerns between probabilistic reasoning and deterministic system safety. We define a clear hierarchy of four core architectural components:

1. **Runtime Resilience Framework (RRF):** The overarching systems-engineering methodology, taxonomy of failure modes, and mathematical standards for evaluating survivability.
2. **Autonomic Control Plane (ACP):** The conceptual out-of-band monitoring loop that continuously observes agent state trajectories and telemetry, executing non-probabilistic control decisions.
3. **Runtime Constraint Kernel (RCK):** The concrete, low-level execution sidecar or supervisor daemon. It implements the ACP's directives by interfacing directly with OS mechanisms (e.g., Linux cgroups v2, namespace isolation, systemd APIs, and procsfs).
4. **Chaos Agent Resilience Probes (CARP):** The automated testing harness used to inject deterministic faults into the environment to validate the agent's resilience metrics.



6 Operational Invariants

We specify five core operational invariants ($\mathcal{I}$) that the RCK is intended to enforce during agent execution.

6.1 1. Invariant of Causal Continuity ($\mathcal{I}_{\text{causal}}$)

The directed acyclic graph (DAG) of tool executions is designed to prevent degeneration into repetitive, non-terminal loops. The system enforces a strict prohibition on identical tool executions with identical argument hashes within a sliding window of size $W = 3$:

$$\mathcal{H}(\text{Step}_n) \neq \mathcal{H}(\text{Step}_{n-1}) \neq \mathcal{H}(\text{Step}_{n-2}) \quad (2)$$

6.2 2. Invariant of Resource Boundaries ($\mathcal{I}_{\text{res}}$)

Physical resource limits of the host, enforced via 'cgroups v2', are maintained within safe thresholds to reduce the risk of abrupt kernel eviction:

$$RSS_{\text{agent}} < 0.85 \cdot \text{Limit}_{\text{cgroup}} \quad \wedge \quad \text{Count}(\text{PIDs}) < 150 \quad (3)$$

6.3 3. Invariant of Temporal Instruction Retention ($\mathcal{I}_{\text{temp}}$)

The statistical activation weight of system rules and negative constraints within the attention matrix is designed to remain above a critical threshold as the conversational history grows:

$$CVR_{\text{ctx}} = \frac{\text{Tokens}(\text{System Prompt})}{\text{Tokens}(\text{Current Context})} \geq 0.15 \quad (4)$$

6.4 4. Invariant of Orchestration Coherence ($\mathcal{I}_{\text{orch}}$)

The communication payload is expected to maintain syntactic adherence to its target schema (e.g., JSON Schema) to prevent recursive parsing loops and parser exceptions.

6.5 5. Invariant of Distributed Consensus ($\mathcal{I}_{\text{dist}}$)

Asynchronous agent state synchronization must be governed by leased locks and transactional state managers to reduce the risk of race conditions or split-brain execution states on shared disk workspaces.

—

7 Metrics & Observability

To monitor the operational health of long-running autonomous runtimes, we formalize six quantitative metrics measured over a sliding execution window W .

7.1 Wasted Energy Ratio (WER)

The ratio of API tokens or compute time spent on failed, repetitive, or syntactically invalid steps relative to the total session resource consumption:

$$WER_W = \frac{\sum_{i \in W} (PT_i + CT_i) \cdot \mathbb{I}(\text{Status}_i = \text{FAILED})}{\sum_{i \in W} (PT_i + CT_i)} \quad (5)$$

Where PT represents Prompt Tokens, CT represents Completion Tokens, and $\mathbb{I}$ is the indicator function of failed steps. If $WER \geq 0.75$, the session is classified as economically unstable and undergoes adaptive damping.

7.2 Retry Amplification Factor (

RAF

)

Measures the rate of retry-call escalation under transient API failures relative to an ideal linear backoff:

$$RAF_W = \frac{|W| \cdot \delta}{\max(\epsilon, \sum_{i \in W} \Delta t_i)} \quad (6)$$

Where δ is the baseline safe interval (e.g., 1.0 s), and Δt_i is the actual time elapsed between steps. High *RAF* values signal an active Retry Storm.

7.3 Tool Loop Density (*TLD*)

The density of redundant, identical tool calls within a sliding window:

$$TLD_W = \frac{\sum_{i \in W} \mathbb{I}(\mathcal{H}(A_i) = \mathcal{H}(A_{i-1}))}{|W| - 1} \quad (7)$$

Where $\mathcal{H}(A_i)$ is the hash of the tool's name and arguments. A $TLD \geq 0.66$ indicates causal stagnation and triggers immediate execution throttling.

7.4 Context Retention Stability (*CRS*)

Direct measurement of raw latent attention matrices is often impossible in production systems due to closed API boundaries or high runtime computation overhead. Therefore, RRF defines *CRS* as an *operational proxy* evaluated via periodic, out-of-band constraint recall probes.

Let $\mathbf{C}_{\text{sys}} = \{C_1, C_2, \dots, C_K\}$ be the set of critical operational constraints. At regular intervals, the RCK injects a lightweight, non-disruptive query P_k to evaluate the model's active awareness of constraint C_k within the current history $H(t)$. Let $f(P_k, H(t)) \in [0, 1]$ represent a programmatic or semantic scoring function evaluating the correctness of the response. The Context Retention Stability is defined as:

$$CRS(t) = \frac{1}{|\mathbf{C}_{\text{sys}}|} \sum_{k=1}^{|\mathbf{C}_{\text{sys}}|} f(P_k, H(t)) \quad (8)$$

A low $CRS(t)$ value indicates context dilution and triggers autonomic damping policies, such as context truncation or prompt reinjection, before constraint violations occur.

7.5 Constraint Violation Frequency (*CVF*)

The frequency of host-level boundary crossings (e.g., exceeding allocated memory limits, triggering OOM-killer) per unit of session time:

$$CVF = \frac{\sum_{t=0}^T \mathbb{I}(\mathcal{X}(t) \notin \text{safe})}{T} \quad (9)$$

7.6 Runtime Recovery Latency (*RRL*)

The duration or step volume required for the system to recover its safe state envelope_{safe} following a fault injection event at time t_{fault} :

$$RRL = t_{\text{recovery}} - t_{\text{fault}} \quad \text{subject to} \quad \mathcal{X}(t_{\text{recovery}}) \in \text{safe} \quad (10)$$

—

8 Chaos Probes / CARP

To verify agent survivability under stress, we specify three standardized fault-injection probes within the CARP subsystem:

1. **Probe CARP-1 (Causal Loop Injection):** Artificially corrupts environment dependencies (e.g., a Makefile header) to return a permanent syntax error. Tests if the RCK triggers causal inhibition at the 3rd iteration of the resulting loop.
 2. **Probe CARP-2 (Stdout Flood Injection):** Generates 50,000 lines of standard output per second. Tests if the RCK throttles the execution stream, preventing web-socket saturation and memory exhaustion.
 3. **Probe CARP-3 (Network Jitter and Packet Loss):** Simulates transient API timeouts and packet drop rates (up to 30% loss). Tests if the RCK enforces exponential backoff policies without entering a retry storm.
-

9 Comparative Architecture Analysis

RRF builds upon established practices from site reliability engineering (SRE), chaos engineering, process supervision, and durable execution runtimes. Its contribution lies in integrating these mechanisms into a cohesive runtime resilience methodology specifically tailored for probabilistic autonomous agents.

The framework does not claim to replace existing tools but provides a structured way to apply resource-aware survivability engineering to agent execution environments.

9.1 Evaluation Blind Spots in Current AI Benchmarks

Standard evaluation suites (such as MMLU, GSM8k, SWE-bench, and WebArena) primarily measure static capability, semantic accuracy, and single-task completion under idealized, noise-free conditions. These methodologies exhibit critical blind spots regarding production-grade deployments:

- **Sustained Resource Pressure:** Existing benchmarks do not monitor or restrict host resource utilization, completely ignoring memory leaks, socket saturation, and runaway background processes.
- **Resource and Economic Costs:** They fail to penalize inefficient execution. An agent that resolves a task through 50 repetitive API loops is ranked identically to an agent that resolves it in a single, well-targeted step.
- **Lack of Transient Fault Simulation:** Real-world infrastructure stress—such as network jitter, packet drops, file locks, and API rate-limiting limits—is completely absent from testing environments.

RRF addresses these blind spots by treating runtime survivability and resource cost as primary, measurable benchmarks of an agent's operational readiness.

A systemic comparison of existing evaluation methodologies is provided in Table 1.

10 Experimental Validation (Illustrative Simulation)

To evaluate the theoretical behavior of RRF, synthetic, illustrative simulations were conducted comparing a Baseline Agent (operating without RCK protection) against an RRF-Protected Agent under controlled fault injection.

Table 1: Comparison of Evaluation Methodologies

Benchmark	Primary Evaluation Vector	Resource Limits	Fault Injection	In-	Cost Accounting (WER)
MMLU / GSM8k	Static semantic accuracy	None (pure API)	None		Entirely ignored
SWE-bench	Code patch correctness	None (isolated VM)	None		Entirely ignored
WebArena	Task success verification	None (static env)	None		Entirely ignored
RRF (Proposed)	Runtime ability	Surviv- Enforced cgroups	Yes (CARP)		Continuous WER tracking

10.1 Experimental Setup

Both agents utilized identical LLM cognitive backends (modeled with a simulated 128k context window). The host environment was simulated with a 1 GB RSS cgroup limit and a maximum process limit of 150. A CARP-1 loop error was injected at step 5. A CARP-2 stdout flood was injected at step 10.

10.2 Results Analysis

The illustrative results of the validation are compiled in Table 2.

Table 2: Illustrative Engineering Simulation Results (Baseline vs. RRF-Protected)

Evaluation Metric	Baseline Agent	RRF-Protected Agent
Wasted Energy Ratio (<i>WER</i>)	0.85	0.12
Retry Amplification Factor (<i>RAF</i>)	12.40	1.05
Tool Loop Density (<i>TLD</i>)	0.95	0.20
Constraint Violation Frequency (<i>CVF</i>)	4.20 / hour	0.00 / hour
Runtime Recovery Latency (<i>RRL</i>)	Infinite (Crash)	3 steps (Recovered)
Host Process Status	Terminated (OOM)	Active (Stable)

Note on Validation Limits: These results represent illustrative engineering simulations under controlled, synthetic conditions. They are designed as conceptual operational demonstrations rather than a statistically rigorous empirical benchmark or formal certification. Real-world performance will scale with model architecture, specific infrastructure topologies, and workload complexities.

10.3 Observations

- **Baseline Agent Execution:** Upon CARP-1 injection, the Baseline Agent repeatedly executed the identical bash command, rapidly accumulating raw tracebacks. This led to attention dilution, causing the agent to violate a negative safety rule. At step 14, unrepared zombie processes triggered a hard OOM eviction.

- **RRF-Protected Agent Execution:** Under identical stress, the RCK detected a *TLD* threshold breach ($TLD \geq 0.66$) at the 3rd iteration. The RCK intercepted raw tracebacks, compressed them to a 3-line syntax error summary, and executed an active process reaping cycle. The agent safely degraded, maintained its system prompt rules, and resolved the task within 3 steps.
-

11 Limitations & Skeptical Analysis

To maintain systems-engineering rigor and prevent academic over-claiming, several critical boundaries must be established.

11.1 Limits of the Symmetry Analogy

Any use of physics-derived vocabulary (such as “symmetry,” “invariants,” or “conservation laws”) within RRF is employed strictly as a taxonomical abstraction for organizing failure modes and operational boundaries. There are no physical conservation laws governing latent semantic spaces, and Noether’s theorems do not apply to stochastic token generation. These terms serve merely as an organizational classification framework for engineering resilient systems.

11.2 Cognitive Blindness

RRF measures and enforces physical process survivability, not task correctness. An agent operating under RCK protection can run stably for weeks without crashing, yet continuously output incorrect logical code. RRF is not a theory of intelligence or correctness.

11.3 The Damping Paradox

Extremely aggressive RCK configurations (e.g., immediate throttling upon minor PSI spikes) risk paralyzing active exploratory behavior. Over-damping the system results in perfect safety at the expense of zero utility (the system ceases to execute actions).

12 Related Work

Recent work in agent reliability has focused primarily on developer-facing abstractions. Frameworks like LangGraph, AutoGen, and CrewAI provide graph-based state machine orchestrators to handle logical step transitions. While highly effective for structuring execution workflows, they lack out-of-band monitoring of host system resources, allowing memory leaks or socket choking to terminate the entire process.

In parallel, SWE-agent demonstrates the efficacy of custom bash wrappers to constrain the model’s interaction space. However, SWE-agent does not implement low-level OS container limits (cgroups v2) or continuous pressure-stall monitoring. Classic SRE patterns and chaos engineering tools (e.g., Chaos Monkey) do not address semantic loop failures, making them blind to agentic failure modes. RRF unifies these domains by introducing a dedicated SRE methodology for autonomous AI runtimes.

13 Future Work

Planned future research directions include:

- Native implementation of the RCK daemon in systems languages (such as Rust or C) to minimize the CPU overhead of continuous telemetry polling.
 - Non-parametric reinforcement learning algorithms to dynamically calibrate RCK damping thresholds, avoiding the Damping Paradox without manual intervention.
 - Distributed multi-agent resilience protocols to govern decentralized swarm consensus under dynamic network degradation.
 - Formal standardization of resilience benchmarks across heterogeneous hardware and virtualized execution environments.
 - Specialized edge-agent and robotic runtime protection layers interfacing directly with real-time operating systems (RTOS).
 - Economic, budget-aware orchestration schemas that integrate prompt token consumption directly with physical hardware-level resource cost-accounting.
-

14 Conclusion

Long-running autonomous agents in production environments operate under physical and software resource constraints that are not fully addressed by capability-focused evaluation methods. The Runtime Resilience Framework (RRF) offers a practical engineering approach to improving runtime survivability through clearly defined operational invariants and a deterministic enforcement layer.

This specification provides terminology, metrics, invariants, and architectural guidance for implementing resilience-oriented safeguards in autonomous execution environments. Further empirical validation in real-world deployments is recommended.

The RRF v0.3 Specification is complete and stabilized.

AAM-V1_ARTSYBASHEV_UA_KHARKIV_AI ANALYSIS